

Práctica 3 del Tema 3

Víctor Manuel Peiró Martínez

FIIS 2ºA

Programación Orientada a Objetos

Enumerado Palo

```
package t3_practica3;
//Enumerado que contiene los palos
public enum Palo {
    OROS,COPAS,ESPADAS,BASTOS
};
```

Clase Carta

```
package t3_practica3;
import java.util.List;
public class Carta implements Comparable<Carta>{

    //Constantes
    public static Integer DEFAULT_NUMERO = 1;
    public static Palo DEFAULT_PALO = Palo. OROS;
    public static Integer DEFAULT_POS = 1;

    //Atributos
    private Integer numero;
    private Palo palo;
    private Integer posicionMazo;

    //Constructores
    //Constructor con 0 atributos
    public Carta() {
        this(Carta.DEFAULT_NUMERO);
    }

    //Constructor con 1 atributos
    public Carta(Integer numero) {
        this(numero,Carta.DEFAULT_PALO);
    }

    //Constructor con 2 atributos
    public Carta (Integer numero, Palo palo) {
        this(numero,palo,Carta.DEFAULT_POS);
    }

    //Constructor Principal
    public Carta(Integer numero, Palo palo, Integer posicionMazo) {
        this.numero = numero;
        this.palo = palo;
        this.posicionMazo = posicionMazo;
    }

    //Getters y Setters
    public Integer getNumero() {
        return this.numero;
    }

    public void setNumero(Integer numero) {
```

```

        this.numero = numero;
    }
    public Palo getPalo() {
        return this.palo;
    }
    public void setPalo(Palo palo) {
        this.palo = palo;
    }
    public Integer getPosicionMazo() {
        return this.posicionMazo;
    }
    public void setPosicionMazo(Integer posicionMazo) {
        this.posicionMazo = posicionMazo;
    }

    //Metodo toString
    public String toString() {
        return "Carta [numero=" + numero + ", palo=" + palo + ", posicionEnMazo=" +
posicionMazo + "]";
    }
    //Compara 2 cartas en funcion de su posicion en el mazo. 1 si es mayor la primera, -1 si es
menor, 0 si es la misma carta
    public int compareTo(Carta carta) {
        int res = 0;

        if (this.posicionMazo > carta.posicionMazo) {
            res = 1;
        } else if (this.posicionMazo < carta.posicionMazo) {
            res = -1;
        }
        return res;
    }

    //Dada un lista de cartas, devuelve la posicion de la carta con el numero mas alto
    public static Integer cartaMasAlta (List<Carta> cartas) {
        Integer indiceCartaAlta=0;

        //Comprueba que la lista no este vacia
        if (cartas == null || cartas.isEmpty()) {
            indiceCartaAlta = -1;
        } else {
            //Compara carta por carta
            for (int i = 0; i < cartas.size(); i++) {
                if (cartas.get(i).getNumero() >
cartas.get(indiceCartaAlta).getNumero()) {
                    indiceCartaAlta = i;
                }
            }
        }
        return indiceCartaAlta;
    }
}

```

Clase Mazo

```
package t3_practica3;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Mazo {

    //Constantes
    public static final Integer MAX_CARTAS = 40;
    public static final Integer DEFAULT_POS = 1;
    public static final List<Carta> DEFAULT_MAZO= new ArrayList<Carta>();

    //Atributos
    private Integer posActual;
    private List<Carta> mazo;

    //Constructor Principal
    public Mazo() {
        //Establecer la posicion a 1 y creacion de la lista de cartas
        this.posActual = Mazo.DEFAULT_POS;
        this.mazo = new ArrayList<Carta>();
        //Se añaden todas las cartas de cada palo
        this.crearPalo(Palo.OROS,this.posActual);
        this.crearPalo(Palo.COPAS,this.mazo.size()+1);
        this.crearPalo(Palo.ESPADAS,this.mazo.size()+1);
        this.crearPalo(Palo.BASTOS,this.mazo.size()+1);
    }

    //Getter y Setters
    public Integer getPosActual() {
        return this.posActual;
    }

    public void setPosActual(Integer posActual) {
        this.posActual = posActual;
    }

    public List<Carta> getMazo() {
        return this.mazo;
    }

    public void setMazo(List<Carta> mazo) {
        this.mazo = mazo;
    }
}
```

```

//Metodo toString
@Override
public String toString() {
    return "Mazo [mazo=" + mazo + "]";
}

//Crea todas las cartas de un palo
private void crearPalo(Palo palo, Integer posicionEnMazo) {
    int i = 1;

    while (i <= 12) {
        Carta carta = new Carta(i,palo,posicionEnMazo);
        this.mazo.add(carta);
        i++;
        posicionEnMazo++;
        //Como no hay 8 ni 9. Hay un salto cuando llegue al 8 para que vaya
al 10
        if (i == 8) {
            i += 2;
        }
    }
}

//Asigna una posicion aleatoria a una carta
private Integer asignaPosicion() {
    Integer posicion = 0;
    int numAleatorio = 0;
    boolean repetido = true;

    //Comprueba que no metamos 2 cartas en la misma posicion
    while (repetido) {
        numAleatorio = (int)((Math.random() * Mazo.MAX_CARTAS)+1);
        //Si la posicion no esta ocupada, se establece posicion y se termina el
bucle
        if (!(this.posicionOcupada(numAleatorio))) {
            posicion = numAleatorio;
            repetido = false;
        }
    }

    return posicion;
}

//Baraja las cartas dandoles posiciones aleatorias
public void barajar() {

    //Establece todas las posiciones a 0 ya las posiciones van de 1 a 40

```

```

        for (int i = 0; i < this.mazo.size(); i++) {
            this.mazo.get(i).setPosicionMazo(Carta.DEFAULT_POS-1);
        }

        //Asigna las posiciones aleatorias
        for (int i = 0; i < this.mazo.size(); i++) {
            this.mazo.get(i).setPosicionMazo(this.asignaPosicion());
        }

        //Ordena el mazo por sus nuevas posiciones y establece la posicion actual al
inicio de mazo
        Collections.sort(this.mazo);
        this.posActual = Mazo.DEFAULT_POS;
    }

    //Comprueba si una posicion esta ya ocupada en el mazo
    private boolean posicionOcupada(Integer posicion) {
        boolean res = false;
        //Busco por todas las cartas del mazo si alguna tiene la misma posicion.
        for (int i = 0; i < this.mazo.size(); i++) {
            if (this.mazo.get(i).getPosicionMazo().equals(posicion)) {
                res = true;
            }
        }
        return res;
    }

    //Veo cuantas cartas hay disponibles
    public Integer cartasDisponibles() {
        return Mazo.MAX_CARTAS - this.posActual + 1;
    }

    //Devuelve la siguiente carta
    public Carta siguienteCarta() {
        Carta siguiente = null;
        if (this.posActual >= 0 && this.posActual <= mazo.size()) {
            siguiente = this.mazo.get(this.posActual-1); //-1 porque es 1 a 40
        }

        return siguiente;
    }

    //Reparte n cartas
    public List<Carta> darCartas(Integer numCartas){
        List<Carta> cartasSolicitadas = new ArrayList<Carta>();
        for (int i = 0; i < numCartas; i++) {
            cartasSolicitadas.add(this.siguienteCarta());
            this.posActual++;
        }
    }

```

```

    }

    return cartasSolicitadas;
}

//Imprime las cartas ya repartidas
public void cartasMonton() {
    for (int i = 0; i < this.posActual-1; i++) {
        System.out.println(this.mazo.get(i));
    }
}

//Muestras las cartas por repartir
public void mostrarBaraja() {
    if (this.posActual <= this.mazo.size()) {
        for (int i = this.posActual-1; i < this.mazo.size(); i++) {
            System.out.println(this.mazo.get(i));
        }
    }
}

//Metodo para probar
public void testMazo() {
    this.mostrarBaraja();
    System.out.println("Barajamos las cartas");
    this.barajar();
    this.mostrarBaraja();
    System.out.println("Cartas disponibles: " +
        this.cartasDisponibles());
    System.out.println("Damos 5 cartas a un jugador");
    List<Carta> reparto1 = this.darCartas(5);
    for(int i=0; i<reparto1.size(); i++) {
        System.out.println(reparto1.get(i));
    }
    System.out.println("Cartas disponibles: " +
        this.cartasDisponibles());
    System.out.println("Mostramos las cartas no repartidas");
    this.mostrarBaraja();
    System.out.println("Sacamos una carta para un jugador" +
        this.darCartas(1));
    System.out.println("Mostramos las 6 cartas que ya han salido");
    this.cartasMonton();
    this.darCartas(this.cartasDisponibles());
    this.mostrarBaraja();
}

```

```
//Main
public static void main(String[] args) {
    Mazo mazo = new Mazo();
    mazo.testMazo();
}
}
```

Ejecución TestMazo

```
<terminated> Mazo [Java Application] C:\Users\Victor Peiro\p2\poo\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_22.0.2.v20240802-1626\jre\bin\javaw.exe (31 oct 2024, 12:45:22) [pid: 18600]
Carta [numero=1, palo=OROS, posicionEnMazo=1]
Carta [numero=2, palo=OROS, posicionEnMazo=2]
Carta [numero=3, palo=OROS, posicionEnMazo=3]
Carta [numero=4, palo=OROS, posicionEnMazo=4]
Carta [numero=5, palo=OROS, posicionEnMazo=5]
Carta [numero=6, palo=OROS, posicionEnMazo=6]
Carta [numero=7, palo=OROS, posicionEnMazo=7]
Carta [numero=10, palo=OROS, posicionEnMazo=8]
Carta [numero=11, palo=OROS, posicionEnMazo=9]
Carta [numero=12, palo=OROS, posicionEnMazo=10]
Carta [numero=1, palo=COPAS, posicionEnMazo=11]
Carta [numero=2, palo=COPAS, posicionEnMazo=12]
Carta [numero=3, palo=COPAS, posicionEnMazo=13]
Carta [numero=4, palo=COPAS, posicionEnMazo=14]
Carta [numero=5, palo=COPAS, posicionEnMazo=15]
Carta [numero=6, palo=COPAS, posicionEnMazo=16]
Carta [numero=7, palo=COPAS, posicionEnMazo=17]
Carta [numero=10, palo=COPAS, posicionEnMazo=18]
Carta [numero=11, palo=COPAS, posicionEnMazo=19]
Carta [numero=12, palo=COPAS, posicionEnMazo=20]
Carta [numero=1, palo=ESPADAS, posicionEnMazo=21]
Carta [numero=2, palo=ESPADAS, posicionEnMazo=22]
Carta [numero=3, palo=ESPADAS, posicionEnMazo=23]
Carta [numero=4, palo=ESPADAS, posicionEnMazo=24]
Carta [numero=5, palo=ESPADAS, posicionEnMazo=25]
Carta [numero=6, palo=ESPADAS, posicionEnMazo=26]
Carta [numero=7, palo=ESPADAS, posicionEnMazo=27]
Carta [numero=10, palo=ESPADAS, posicionEnMazo=28]
Carta [numero=11, palo=ESPADAS, posicionEnMazo=29]
Carta [numero=12, palo=ESPADAS, posicionEnMazo=30]
Carta [numero=1, palo=BASTOS, posicionEnMazo=31]
Carta [numero=2, palo=BASTOS, posicionEnMazo=32]
Carta [numero=3, palo=BASTOS, posicionEnMazo=33]
Carta [numero=4, palo=BASTOS, posicionEnMazo=34]
Carta [numero=5, palo=BASTOS, posicionEnMazo=35]
Carta [numero=6, palo=BASTOS, posicionEnMazo=36]
Carta [numero=7, palo=BASTOS, posicionEnMazo=37]
Carta [numero=10, palo=BASTOS, posicionEnMazo=38]
Carta [numero=11, palo=BASTOS, posicionEnMazo=39]
Carta [numero=12, palo=BASTOS, posicionEnMazo=40]

<terminated> Mazo [Java Application] C:\Users\Victor Peiro\p2\poo\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_22.0.2.v20240802-1626\jre\bin\javaw.exe (31 oct 2024, 12:45:22) [pid: 18600]
Barajamos las cartas
Carta [numero=5, palo=BASTOS, posicionEnMazo=1]
Carta [numero=10, palo=OROS, posicionEnMazo=2]
Carta [numero=12, palo=BASTOS, posicionEnMazo=3]
Carta [numero=4, palo=ESPADAS, posicionEnMazo=4]
Carta [numero=11, palo=OROS, posicionEnMazo=5]
Carta [numero=12, palo=OROS, posicionEnMazo=6]
Carta [numero=4, palo=COPAS, posicionEnMazo=7]
Carta [numero=3, palo=ESPADAS, posicionEnMazo=8]
Carta [numero=6, palo=BASTOS, posicionEnMazo=9]
Carta [numero=11, palo=COPAS, posicionEnMazo=10]
Carta [numero=10, palo=COPAS, posicionEnMazo=11]
Carta [numero=5, palo=ESPADAS, posicionEnMazo=12]
Carta [numero=1, palo=ESPADAS, posicionEnMazo=13]
Carta [numero=2, palo=ESPADAS, posicionEnMazo=14]
Carta [numero=6, palo=OROS, posicionEnMazo=15]
Carta [numero=10, palo=ESPADAS, posicionEnMazo=16]
Carta [numero=7, palo=BASTOS, posicionEnMazo=17]
Carta [numero=1, palo=COPAS, posicionEnMazo=18]
Carta [numero=12, palo=COPAS, posicionEnMazo=19]
Carta [numero=1, palo=BASTOS, posicionEnMazo=20]
Carta [numero=4, palo=BASTOS, posicionEnMazo=21]
Carta [numero=3, palo=COPAS, posicionEnMazo=22]
Carta [numero=2, palo=OROS, posicionEnMazo=23]
Carta [numero=6, palo=COPAS, posicionEnMazo=24]
Carta [numero=4, palo=OROS, posicionEnMazo=25]
Carta [numero=1, palo=OROS, posicionEnMazo=26]
Carta [numero=3, palo=BASTOS, posicionEnMazo=27]
Carta [numero=7, palo=COPAS, posicionEnMazo=28]
Carta [numero=10, palo=BASTOS, posicionEnMazo=29]
Carta [numero=2, palo=COPAS, posicionEnMazo=30]
Carta [numero=11, palo=ESPADAS, posicionEnMazo=31]
Carta [numero=11, palo=BASTOS, posicionEnMazo=32]
Carta [numero=3, palo=OROS, posicionEnMazo=33]
Carta [numero=2, palo=BASTOS, posicionEnMazo=34]
Carta [numero=12, palo=ESPADAS, posicionEnMazo=35]
Carta [numero=7, palo=ESPADAS, posicionEnMazo=36]
Carta [numero=7, palo=OROS, posicionEnMazo=37]
Carta [numero=5, palo=OROS, posicionEnMazo=38]
Carta [numero=6, palo=ESPADAS, posicionEnMazo=39]
Carta [numero=5, palo=COPAS, posicionEnMazo=40]
```



```
Cartas disponibles: 40
Damos 5 cartas a un jugador
Carta [numero=5, palo=BASTOS, posicionEnMazo=1]
Carta [numero=10, palo=OROS, posicionEnMazo=2]
Carta [numero=12, palo=BASTOS, posicionEnMazo=3]
Carta [numero=4, palo=ESPADAS, posicionEnMazo=4]
Carta [numero=11, palo=OROS, posicionEnMazo=5]
Cartas disponibles: 35
Mostramos las cartas no repartidas
Carta [numero=12, palo=OROS, posicionEnMazo=6]
Carta [numero=4, palo=COPAS, posicionEnMazo=7]
Carta [numero=3, palo=ESPADAS, posicionEnMazo=8]
Carta [numero=6, palo=BASTOS, posicionEnMazo=9]
Carta [numero=11, palo=COPAS, posicionEnMazo=10]
Carta [numero=10, palo=COPAS, posicionEnMazo=11]
Carta [numero=5, palo=ESPADAS, posicionEnMazo=12]
Carta [numero=1, palo=ESPADAS, posicionEnMazo=13]
Carta [numero=2, palo=ESPADAS, posicionEnMazo=14]
Carta [numero=6, palo=OROS, posicionEnMazo=15]
Carta [numero=10, palo=ESPADAS, posicionEnMazo=16]
Carta [numero=7, palo=BASTOS, posicionEnMazo=17]
Carta [numero=1, palo=COPAS, posicionEnMazo=18]
Carta [numero=12, palo=COPAS, posicionEnMazo=19]
Carta [numero=1, palo=BASTOS, posicionEnMazo=20]
Carta [numero=4, palo=BASTOS, posicionEnMazo=21]
Carta [numero=3, palo=COPAS, posicionEnMazo=22]
Carta [numero=2, palo=OROS, posicionEnMazo=23]
Carta [numero=6, palo=COPAS, posicionEnMazo=24]
Carta [numero=4, palo=OROS, posicionEnMazo=25]
Carta [numero=1, palo=OROS, posicionEnMazo=26]
Carta [numero=3, palo=BASTOS, posicionEnMazo=27]
Carta [numero=7, palo=COPAS, posicionEnMazo=28]
Carta [numero=10, palo=BASTOS, posicionEnMazo=29]
Carta [numero=2, palo=COPAS, posicionEnMazo=30]
Carta [numero=11, palo=ESPADAS, posicionEnMazo=31]
Carta [numero=11, palo=BASTOS, posicionEnMazo=32]
Carta [numero=3, palo=OROS, posicionEnMazo=33]
Carta [numero=2, palo=BASTOS, posicionEnMazo=34]
Carta [numero=12, palo=ESPADAS, posicionEnMazo=35]
Carta [numero=7, palo=ESPADAS, posicionEnMazo=36]
Carta [numero=7, palo=OROS, posicionEnMazo=37]
Carta [numero=5, palo=OROS, posicionEnMazo=38]
Carta [numero=6, palo=ESPADAS, posicionEnMazo=39]
Carta [numero=5, palo=COPAS, posicionEnMazo=40]
```

Sacamos una carta para un jugador[Carta [numero=12, palo=OROS, posicionEnMazo=6]]

Mostramos las 6 cartas que ya han salido

```
Carta [numero=5, palo=BASTOS, posicionEnMazo=1]
Carta [numero=10, palo=OROS, posicionEnMazo=2]
Carta [numero=12, palo=BASTOS, posicionEnMazo=3]
Carta [numero=4, palo=ESPADAS, posicionEnMazo=4]
Carta [numero=11, palo=OROS, posicionEnMazo=5]
Carta [numero=12, palo=OROS, posicionEnMazo=6]
```

Clase Jugador

```
package t3_practica3;
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
public class Jugador implements Comparable<Jugador>{
```

```
    //Constantes
```

```
    public static final String DEFAULT_NAME = "Jugador";
```

```
    public static final Integer DEFAULT_PUNTOS_TOTAL = 0;
```

```
    public static final Integer DEFAULT_TURN0 = 0;
```

```
    //Atributos
```

```
    private String nombre;
```

```
    private List<Carta> cartas;
```

```
    private List<Integer> puntosPorRonda;
```

```
    private Integer puntosTotal;
```

```
    private Integer turno;
```

```
    //Constructores
```

```
    //Constructor con 0 atributos
```

```
    public Jugador() {
```

```
        this(Jugador.DEFAULT_NAME);
```

```
    }
```

```
    //Constructor con 1 atributo
```

```
    public Jugador(String nombre) {
```

```
        this(nombre,new ArrayList<Integer>());
```

```
    }
```

```
    //Constructor con 2 atributos
```

```
    public Jugador(List<Integer> puntosPorRonda) {
```

```
        this(Jugador.DEFAULT_NAME,puntosPorRonda);
```

```
    }
```

```
    //Constructor con 3 atributos
```

```
    public Jugador(String nombre, List<Integer> puntosPorRonda) {
```

```
        this(nombre,puntosPorRonda,Jugador.DEFAULT_TURN0);
```

```
    }
```

```
    //Constructor Principal
```

```
    public Jugador(String nombre, List<Integer> puntosPorRonda, Integer turno) {
```

```
        this.nombre = nombre;
```

```
        this.puntosPorRonda = puntosPorRonda;
```

```
        this.turno = turno;
```

```

        this.puntosTotal = Jugador.DEFAULT_PUNTOS_TOTAL;
        this.cartas = new ArrayList<Carta>();
    }

    //Getters y Setters
    public String getNombre() {
        return this.nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public List<Carta> getCartas() {
        return this.cartas;
    }

    public void setCartas(List<Carta> cartas) {
        this.cartas = cartas;
    }

    public List<Integer> getPuntosPorRonda() {
        return this.puntosPorRonda;
    }

    public void setPuntosPorRonda(List<Integer> puntosPorRonda) {
        this.puntosPorRonda = puntosPorRonda;
    }

    public Integer getPuntosTotal() {
        return this.puntosTotal;
    }

    public void setPuntosTotal(Integer puntosTotal) {
        this.puntosTotal = puntosTotal;
    }

    public Integer getTurno() {
        return this.turno;
    }

    public void setTurno(Integer turno) {
        this.turno = turno;
    }

    //Metodo toString
    public String toString() {

```

```

        return "Jugador [nombre=" + nombre + ", cartas=" + cartas + ",
puntosPorRonda=" + puntosPorRonda
                + ", puntosTotal=" + puntosTotal + ", turno=" + turno + "];
    }

    //Compara 2 jugadores por su orden
    public int compareTo(Jugador jugador2) {
        int res = 0;

        if (this.turno > jugador2.getTurno()) {
            res = 1;
        } else if (this.turno < jugador2.getTurno()){
            res = -1;
        }
        return res;
    }

    //Escoge una carta de las cartas de jugador y la devuelve eliminandola de la lista
    public Carta echarCarta(){
        Integer posCarta = (int)(Math.random() * this.cartas.size());
        Carta cartaElegida = this.cartas.get(posCarta);
        this.cartas.remove(cartaElegida);
        return cartaElegida;
    }

    //Añade una carta a las cartas del jugador
    public void meterCarta(Carta cartaNueva) {
        this.cartas.add(cartaNueva);
    }

    //Añade la puntuacion de esa ronda
    public void establecerPuntuacion(Integer puntuacion) {
        this.puntosPorRonda.add(puntuacion);
    }

    //Suma las puntuaciones de todas las rondas
    public void calcularPuntosTotal() {
        this.puntosTotal = 0;
        for (int i = 0; i < this.puntosPorRonda.size(); i++) {
            this.puntosTotal += this.puntosPorRonda.get(i);
        }
    }
}

```

Clase PartidaMaximos

```
package t3_practica3;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class PartidaMaximos {

    //Atributos
    private List<Jugador> jugadores;
    private Mazo baraja;

    //Constructor Principal
    public PartidaMaximos(List<Jugador> jugadores) {
        this.jugadores = jugadores;
        //Crea el mazo y lo baraja
        this.baraja = new Mazo();
        this.baraja.barajar();

        //Asigna un orden de turno a cada jugador
        for (int i = 0; i < this.jugadores.size(); i++) {
            this.jugadores.get(i).setTurno(i);
        }

        //Reparte las cartas equitativamemnte
        this.repartirCartas();
    }

    //Reparte las cartas
    private void repartirCartas() {
        Integer cartasPorJugador = this.baraja.getMazo().size() /
this.jugadores.size(); //Calcula cuantas cartas se dan por jugador
        for (int i = 0; i < this.jugadores.size(); i++) {

this.jugadores.get(i).setCartas(this.baraja.darCartas(cartasPorJugador));
        }
    }

    //Simula un ronda
    private void jugarRonda() {
        List<Carta> cartasEchadas = new ArrayList<Carta>();
        for(int i = 0; i < this.jugadores.size(); i++) {
            cartasEchadas.add(this.jugadores.get(i).echarCarta());
        }
        this.establecerGanadorRonda(cartasEchadas);
    }
}
```

```

    }

    //Simula todas las rondas
    public void jugarPartida() {
        Integer numRonda = 0;
        //Hasta que el primer jugador se quede sin cartas --> Se sigue jugando
        while (this.jugadores.get(0).getCartas().size() > 0) {
            numRonda++;
            System.out.println("Ronda: " + numRonda);
            this.jugarRonda(); //Se juega la ronda
            this.calcularMarcador(); //Se calcula en marcador
            this.cambiarOrdenTurno(); //Se cambia en orden de los turnos
        }
        //Imprime el ganador
        System.out.println("Partida Terminada: ");
        System.out.println("El ganador es: " + this.buscarGanador().getNombre());
    }

    //Calcular Marcador
    private void calcularMarcador() {
        System.out.println("Resultado: ");
        for (int i = 0; i < this.jugadores.size(); i++) {
            this.jugadores.get(i).calcularPuntosTotal();
            System.out.println(this.jugadores.get(i).getNombre() + " --> " +
this.jugadores.get(i).getPuntosTotal());
        }
    }

    //Se busca el ganador
    public Jugador buscarGanador() {
        Jugador ganador = this.jugadores.get(0);

        for (int i = 0; i < this.jugadores.size(); i++) {
            if (this.jugadores.get(i).getPuntosTotal() > ganador.getPuntosTotal()) {
                ganador = this.jugadores.get(i);
            }
        }

        return ganador;
    }

    //Ve quien a ganado la ronda y le da un punto
    private void establecerGanadorRonda(List<Carta> cartasEchadas) {
        Integer posCartaGanadora = Carta.cartaMasAlta(cartasEchadas); //Comprueba la
carta mas alta
        //Imprime el resultado
        System.out.println("La carta ganadora es: " +
cartasEchadas.get(posCartaGanadora));
    }

```

```

        System.out.println("Ganador: " +
this.jugadores.get(posCartaGanadora).getNombre());
        this.jugadores.get(posCartaGanadora).establecerPuntuacion(1);
    }

    //Cambia el orden de los turnos --> el segundo pasa a primero, tercero a segundo,
    etc..
    private void cambiarOrdenTurno() {
        for (int i = 0; i < this.jugadores.size(); i++) {
            this.jugadores.get(i).setTurno((i-1));
            if (i-1 < 0) {
                this.jugadores.get(i).setTurno((this.jugadores.size()-1));
            }
        }
        //Ordena a los jugadores en funcion de su nuevo turno
        Collections.sort(this.jugadores);
    }

    //Main
    public static void main(String[] args) {
        Jugador jugador1 = new Jugador("Laura");
        Jugador jugador2 = new Jugador("Sergio");
        Jugador jugador3 = new Jugador("Adrián");
        Jugador jugador4 = new Jugador("Saray");

        List<Jugador> jugadores = new ArrayList<Jugador>();
        jugadores.add(jugador1);
        jugadores.add(jugador2);
        jugadores.add(jugador3);
        jugadores.add(jugador4);

        PartidaMaximos partida = new PartidaMaximos(jugadores);
        partida.jugarPartida();
    }
}

```

Ejecución

```
<terminated> PartidaMaximos [Java Application] C:\Users\Victor Peiro\.p2\pool\plugins\org.eclipse.ju
Ronda: 1
La carta ganadora es: Carta [numero=10, palo=COPAS, posicionEnMazo=32]
Ganador: Saray
Resultado:
Laura --> 0
Sergio --> 0
Adrián --> 0
Saray --> 1
Ronda: 2
La carta ganadora es: Carta [numero=11, palo=ESPADAS, posicionEnMazo=39]
Ganador: Saray
Resultado:
Sergio --> 0
Adrián --> 0
Saray --> 2
Laura --> 0
Ronda: 3
La carta ganadora es: Carta [numero=12, palo=BASTOS, posicionEnMazo=34]
Ganador: Saray
Resultado:
Adrián --> 0
Saray --> 3
Laura --> 0
Sergio --> 0
Ronda: 4
La carta ganadora es: Carta [numero=10, palo=ESPADAS, posicionEnMazo=28]
Ganador: Adrián
Resultado:
Saray --> 3
Laura --> 0
Sergio --> 0
Adrián --> 1
Ronda: 5
La carta ganadora es: Carta [numero=12, palo=OROS, posicionEnMazo=27]
Ganador: Adrián
Resultado:
Laura --> 0
Sergio --> 0
Adrián --> 2
Saray --> 3
```


<terminated> PartidaMaximos [Java Application] C:\Users\Victor Peiro\.p2\pool\plugins\org.eclipse.justj.o

Ronda: 6

La carta ganadora es: Carta [numero=12, palo=ESPADAS, posicionEnMazo=19]

Ganador: Sergio

Resultado:

Sergio --> 1

Adrián --> 2

Saray --> 3

Laura --> 0

Ronda: 7

La carta ganadora es: Carta [numero=11, palo=COPAS, posicionEnMazo=24]

Ganador: Adrián

Resultado:

Adrián --> 3

Saray --> 3

Laura --> 0

Sergio --> 1

Ronda: 8

La carta ganadora es: Carta [numero=12, palo=COPAS, posicionEnMazo=11]

Ganador: Sergio

Resultado:

Saray --> 3

Laura --> 0

Sergio --> 2

Adrián --> 3

Ronda: 9

La carta ganadora es: Carta [numero=7, palo=COPAS, posicionEnMazo=2]

Ganador: Laura

Resultado:

Laura --> 1

Sergio --> 2

Adrián --> 3

Saray --> 3

Ronda: 10

La carta ganadora es: Carta [numero=7, palo=ESPADAS, posicionEnMazo=5]

Ganador: Laura

Resultado:

Sergio --> 2

Adrián --> 3

Saray --> 3

Laura --> 2

Partida Terminada:

El ganador es: Adrián